

COMPREHENSIVE GUIDE ON SSH TUNNELING





Contents

Introduction	4
Part 1: Setting Up the Target Web Server	4
Installing Apache2 on the Target Machine.....	4
Creating a Custom Web Directory and HTML File.....	4
Configuring Apache2 to Listen Only on Loopback Port 8080.....	5
Configuring the Apache2 Virtual Host	5
Enabling the Site and Restarting Apache2	6
Verifying Local Access on the Target Machine	6
Confirming the Service is Not Remotely Accessible.....	7
Part 2: Local Port Forwarding.....	8
Verifying Port 8000 is Closed Before Tunnelling	8
Establishing the SSH Local Port Forward.....	9
Running the Tunnel in the Background.....	9
Verifying the Tunnel with Nmap	10
Accessing the Remote Service Through the Tunnel.....	10
Verifying the Active Tunnel with Netstat	11
Part 3: Dynamic Port Forwarding (SOCKS Proxy)	11
Establishing the Dynamic Port Forward	11
Verifying the SOCKS Proxy with Netstat.....	12
Configuring Proxychains.....	12
Launching Firefox Through Proxychains.....	13
Accessing the Internal Service via Dynamic Proxy	13
Part 4: Remote Port Forwarding	14
Establishing the SSH Remote Port Forward	14
Verifying the Remote Forward with Netstat on the SSH Server	15
Part 5: RDP Access via Dynamic Port Forwarding and Proxychains	16
Establishing the SSH Dynamic Tunnel from Kali.....	16
Launching rdesktop Through Proxychains	17
Windows RDP Login Screen via Proxychains.....	17
Accessing the Windows Desktop	18
Part 6: RDP Access via SSH Remote Port Forwarding from the Pivot Host	18
Initiating Remote Port Forward from Ubuntu to Kali.....	18
Verifying the Remote Forward on Kali with Netstat	19
Accessing RDP Directly on Kali's Loopback	19





Part 7: RDP Access via GatewayPorts — Exposing the Tunnel to the Network	20
Checking the Default GatewayPorts Setting	20
Enabling GatewayPorts	21
Restarting the SSH Service	22
Initiating Remote Port Forward from Windows to Ubuntu	22
Direct RDP Connection from Kali Using the Network IP	23
Full Windows Desktop via GatewayPorts RDP Tunnel	23
Conclusion.....	24
Mitigation Strategies	25



This article walks through SSH tunnelling in a practical, lab-oriented way. You will see how to set up a loopback-bound Apache2 web server as a protected target, then use Local, Dynamic, and Remote Port Forwarding to reach it in different ways. You will also explore RDP tunnelling with proxychains and learn how the GatewayPorts option exposes remote forwards to an entire network. Each section includes clear commands, configuration steps, and verification methods so you can safely reproduce the techniques in authorized environments.

Introduction

In cybersecurity today, knowing how to use core network protocols for both attack and defence is essential for any serious practitioner. Secure Shell (SSH) is a perfect example. Most people think of SSH as a secure way to log into remote systems, and it certainly is, which is why administrators, engineers, and developers rely on it every day. However, SSH also includes powerful port-forwarding features that turn it into a flexible tunnelling tool. With the right configuration, SSH can securely pass traffic through firewalls, cross NAT boundaries, and provide encrypted access to internal services that are not directly reachable.

Part 1: Setting Up the Target Web Server

Installing Apache2 on the Target Machine

The first step is to install the Apache2 web server on the Ubuntu target machine. And for this, use the following command:

```
apt install apache2
```

```
root@ignite:~# apt install apache2
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages were automatically installed and are
  libflashrom1 libftdi1-2 libllvm13
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
```

Creating a Custom Web Directory and HTML File

With Apache2 installed, the next step is to create a dedicated directory and a simple HTML landing page that will serve as the web application we intend to access through the tunnel. We will then navigate to this said directory and create a webpage. For this, use the following commands :



```
mkdir /sbin/test  
cd /sbin/test/  
echo "Welcome to Hacking Articles" > index.html
```

```
root@ignite:~# mkdir /sbin/test  
root@ignite:~#  
root@ignite:~# cd /sbin/test/  
root@ignite:/sbin/test#  
root@ignite:/sbin/test# echo "Welcome to Hacking Articles" > index.html  
root@ignite:/sbin/test#  
root@ignite:/sbin/test#
```

Configuring Apache2 to Listen Only on Loopback Port 8080

Make sure the `ports.conf` file has been modified to include **Listen 127.0.0.1:8080** — binding Apache exclusively to the loopback interface on port 8080. The standard `Listen 80` directive is also present for default traffic, along with SSL listeners on port 443. The loopback binding ensures that no external machine can directly reach this service over the network.

```
root@ignite:/etc/apache2# cat ports.conf  
# If you just change the port or add more ports here, you will likely also  
# have to change the VirtualHost statement in  
# /etc/apache2/sites-enabled/000-default.conf  
  
Listen 127.0.0.1:8080  
Listen 80  
  
<IfModule ssl_module>  
    Listen 443  
</IfModule>  
  
<IfModule mod_gnutls.c>  
    Listen 443  
</IfModule>  
  
# vim: syntax=apache ts=4 sw=4 sts=4 sr noet  
root@ignite:/etc/apache2#  
root@ignite:/etc/apache2#
```

Configuring the Apache2 Virtual Host

A VirtualHost configuration file is created to define how Apache2 serves requests arriving at 127.0.0.1:8080. This configuration specifies the document root, server name, URL encoding behavior, and directory access permissions. Use the following command for this:

```
nano /etc/apache2/sites-available/test.conf
```

```
<VirtualHost 127.0.0.1:8080>
```

```
DocumentRoot /sbin/test/
```

```
ServerName localhost
```

```
AllowEncodedSlashes NoDecode
```





```
<Directory "/sbin/test/">
```

```
Require all granted
```

```
AllowOverride All
```

```
Options FollowSymLinks MultiViews
```

```
</Directory>
```

```
</VirtualHost>
```

```
root@ignite:/etc/apache2# cat /etc/apache2/sites-available/test.conf ←
<VirtualHost 127.0.0.1:8080>
  DocumentRoot /sbin/test/
  ServerName localhost
  AllowEncodedSlashes NoDecode
  <Directory "/sbin/test/">
    Require all granted
    AllowOverride All
    Options FollowSymLinks MultiViews
  </Directory>
</VirtualHost>
```

Enabling the Site and Restarting Apache2

The previously created virtual host configuration must be activated using the following command:

```
a2ensite test.conf
```

The above command will be paired with another command to reload Apache web server with the newly applied configuration :

```
systemctl restart apache2
```

```
root@ignite:/etc/apache2# a2ensite test.conf ←
Enabling site test.
To activate the new configuration, you need to run:
  systemctl reload apache2
root@ignite:/etc/apache2# systemctl restart apache2 ←
root@ignite:/etc/apache2#
```

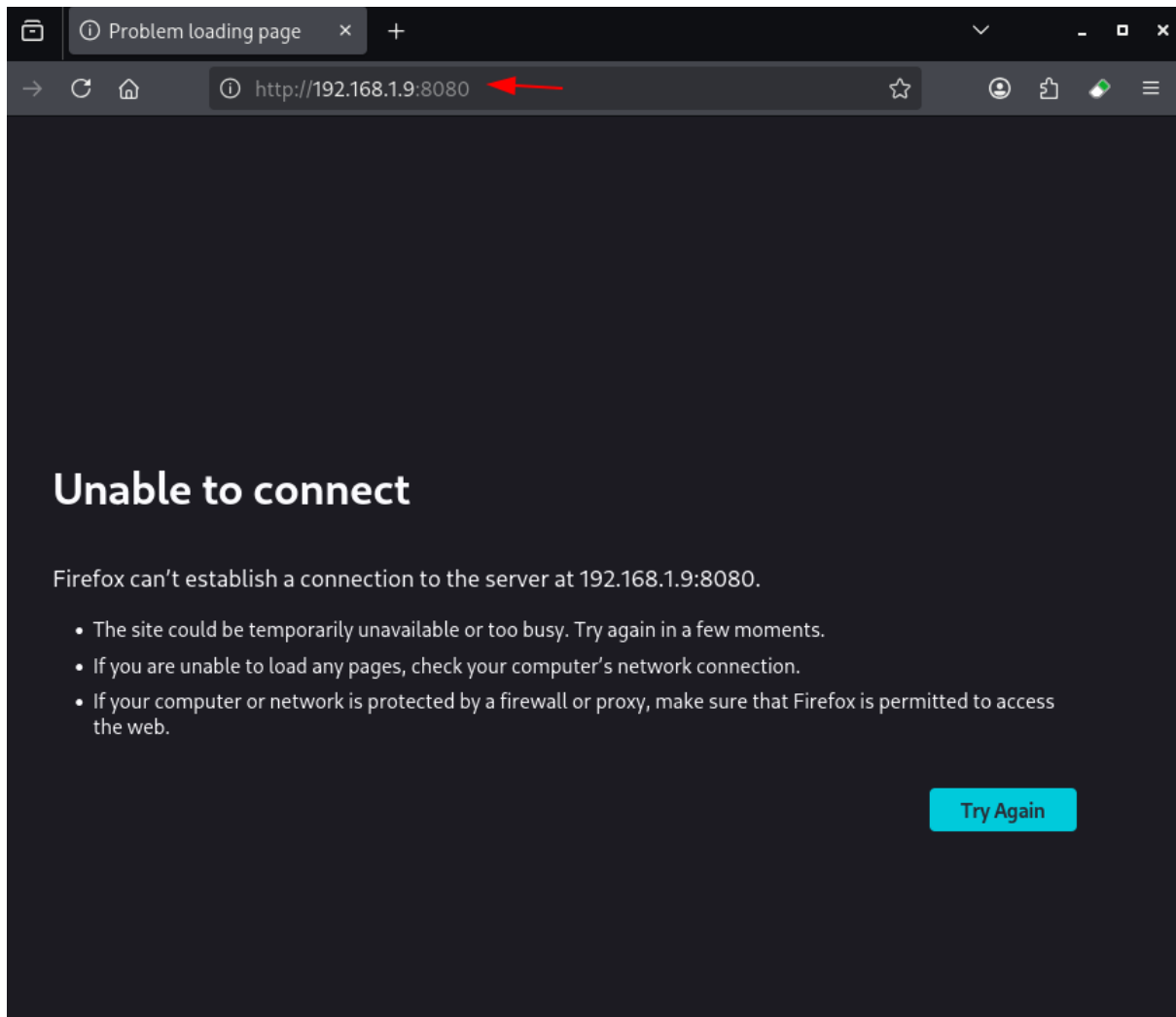
Verifying Local Access on the Target Machine

Navigating to `127.0.0.1:8080` in a browser on the target confirms that Apache2 is serving the page correctly.



Confirming the Service is Not Remotely Accessible

When attempting to access the Apache2 service from the remote Kali Linux machine using the target's actual network IP address (192.168.1.9) and port 8080. Firefox returns an "Unable to connect" error. Since Apache2 is bound only to 127.0.0.1 (the loopback interface), any connection attempt from a different network address is refused. This is precisely the scenario that SSH port forwarding is designed to overcome.



Part 2: Local Port Forwarding

Local port forwarding is the most fundamental form of SSH tunnelling. It allows a port on the local (client) machine to be forwarded through an SSH connection to a specific port on the remote (server) machine. This technique is ideal for accessing services that are bound to the loopback address on a remote host.

Verifying Port 8000 is Closed Before Tunnelling

Perform the following nmap scan on the local machine to confirm that port 8000 — the port we plan to use locally — is not yet in use.

```
nmap -p 8000 127.0.0.1
```



```
root@kali ~  
➤ nmap -p 8000 127.0.0.1  
  
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-09 16:10 +0530  
Nmap scan report for localhost (127.0.0.1)  
Host is up (0.000049s latency).  
  
PORT      STATE SERVICE  
8000/tcp  closed http-alt  
  
Nmap done: 1 IP address (1 host up) scanned in 0.11 seconds  
root@kali ~
```

Establishing the SSH Local Port Forward

The following command instructs SSH to listen on local port 8000 and forward all incoming traffic through the SSH connection to port 8080 on the remote host's loopback interface.

```
ssh -L 127.0.0.1:8000:127.0.0.1:8080 pentest@192.168.1.9
```

```
root@kali ~  
➤ ssh -L 127.0.0.1:8000:127.0.0.1:8080 pentest@192.168.1.9  
pentest@192.168.1.9's password:  
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-106-generic x86_64)  
  
* Documentation:  https://help.ubuntu.com  
* Management:    https://landscape.canonical.com  
* Support:       https://ubuntu.com/pro  
  
Expanded Security Maintenance for Applications is not enabled.  
  
0 updates can be applied immediately.  
  
6 additional security updates can be applied with ESM Apps.  
Learn more about enabling ESM Apps service at https://ubuntu.com/esm  
  
New release '24.04.4 LTS' available.  
Run 'do-release-upgrade' to upgrade to it.  
  
*** System restart required ***  
Last login: Thu Apr  9 16:08:50 2026 from 192.168.1.17  
pentest@ignite:~$
```

After authentication, an SSH session is established with the Ubuntu target, and local port 8000 on the Kali host is transparently forwarded to port 8080 on the target's loopback interface.

Running the Tunnel in the Background

For non-interactive scenarios, the SSH tunnel can be launched silently in the background with the following command:

```
ssh -f -N -L 127.0.0.1:8000:127.0.0.1:8080 pentest@192.168.1.9
```





```
root@kali ~  
└─> ssh -f -N -L 127.0.0.1:8000:127.0.0.1:8080 pentest@192.168.1.9  
pentest@192.168.1.9's password:  
root@kali ~
```

After authentication, the prompt returns immediately while the tunnel continues to run silently in the background, making this approach well suited to automated workflows and long-lived tunnels.

Verifying the Tunnel with Nmap

With the tunnel established, a follow-up Nmap scan on the local host verifies that port 8000 is now open and listening. The said nmap command is:

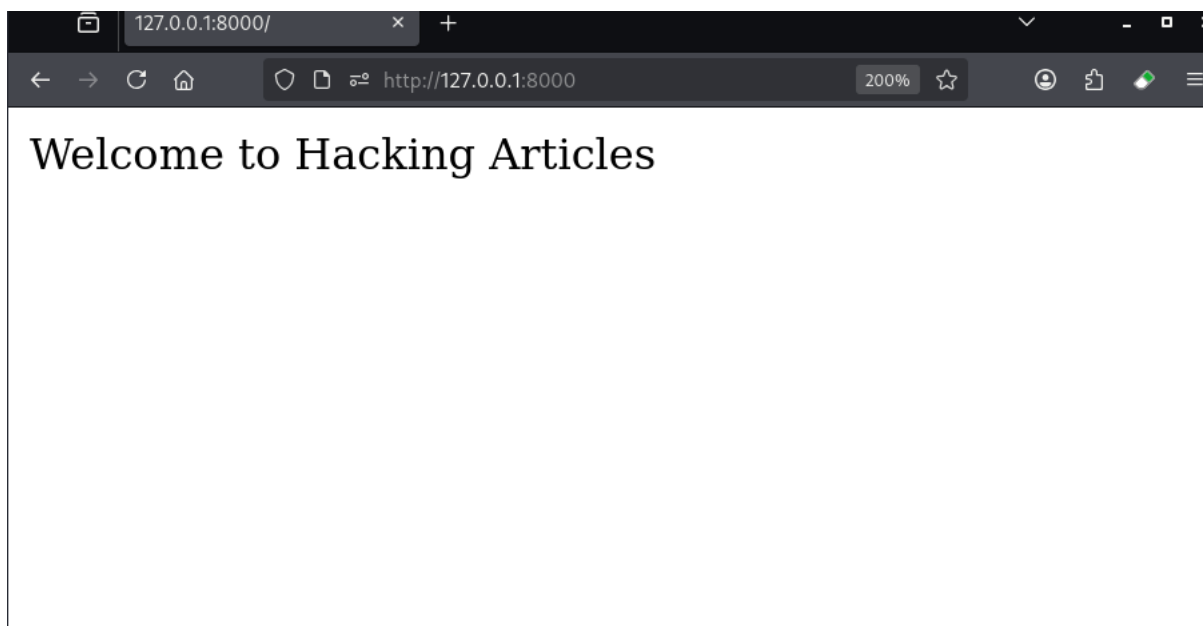
```
nmap -p 8000 127.0.0.1
```

```
root@kali ~  
└─> nmap -p 8000 127.0.0.1  
  
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-09 16:12 +0530  
Nmap scan report for localhost (127.0.0.1)  
Host is up (0.000047s latency).  
  
PORT      STATE SERVICE  
8000/tcp  open  http-alt  
  
Nmap done: 1 IP address (1 host up) scanned in 0.10 seconds  
root@kali ~
```

The port is now actively listening on the local host, confirming that the SSH tunnel is live and ready to relay traffic to the remote Apache2 service.

Accessing the Remote Service Through the Tunnel

Browsing to <http://127.0.0.1:8000> on the Kali host now serves content from the remote Apache2 instance bound to 127.0.0.1:8080 on the target. This demonstrates that, despite the service being restricted to the remote loopback interface, SSH local port forwarding seamlessly proxies the connection through an encrypted tunnel.



Verifying the Active Tunnel with Netstat

Executing the following command on the Kali machine provides a detailed view of all currently listening TCP ports, confirming the active tunnel and the SSH process responsible for it.

```
netstat -tnlp
```

```
root@kali ~  
└─$ netstat -tnlp  
Active Internet connections (only servers)  
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name  
tcp        0      0 127.0.0.1:46175         0.0.0.0:*               LISTEN      984/containerd  
tcp        0      0 0.0.0.0:22             0.0.0.0:*               LISTEN      12999/sshd: /usr/sb  
tcp        0      0 127.0.0.1:8000         0.0.0.0:*               LISTEN      22420/ssh  
tcp6       0      0 :::22                  :::*                    LISTEN      12999/sshd: /usr/sb
```

This provides definitive proof that local port forwarding is active and functional.

Part 3: Dynamic Port Forwarding (SOCKS Proxy)

Dynamic port forwarding launches a local SOCKS proxy that routes traffic through the SSH connection. Unlike local port forwarding, which targets a single endpoint, it can reach any host and port, effectively turning the SSH server into a flexible proxy gateway. This makes it especially useful for directing all browser traffic through a remote network.

Establishing the Dynamic Port Forward

The SSH dynamic port forwarding command creates a SOCKS proxy on local port 8000, routing all proxy traffic through the SSH connection to the target machine.

```
ssh -D 127.0.0.1:8000 pentest@192.168.1.9
```



```
root@kali ~  
└─> ssh -D 127.0.0.1:8000 pentest@192.168.1.9  
pentest@192.168.1.9's password:  
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-107-generic x86_64)  
  
* Documentation:  https://help.ubuntu.com  
* Management:    https://landscape.canonical.com  
* Support:        https://ubuntu.com/pro  
  
Expanded Security Maintenance for Applications is not enabled.  
  
0 updates can be applied immediately.  
  
6 additional security updates can be applied with ESM Apps.  
Learn more about enabling ESM Apps service at https://ubuntu.com/esm  
  
New release '24.04.4 LTS' available.  
Run 'do-release-upgrade' to upgrade to it.  
  
Last login: Fri Apr 10 14:38:43 2026 from 192.168.1.17  
pentest@ignite:~$  
pentest@ignite:~$
```

Any application configured to use this SOCKS proxy will have its traffic tunnelled through the SSH connection to the target machine.

Verifying the SOCKS Proxy with Netstat

Netstat confirms that the dynamic port forward is active, with port 8000 on the loopback interface now listening under the SSH process.

```
netstat -tnlp
```

```
root@kali ~  
└─> netstat -tnlp  
Active Internet connections (only servers)  
Proto Recv-Q Send-Q Local Address      Foreign Address    State       PID/Program name  
tcp        0      0 127.0.0.1:8000      0.0.0.0:*          LISTEN      3102/ssh  
tcp        0      0 127.0.0.1:36941     0.0.0.0:*          LISTEN      744/containerd
```

This confirms the SOCKS proxy is ready to receive and forward application traffic through the encrypted SSH tunnel.

Configuring Proxychains

To route application traffic through the SOCKS proxy, the `proxychains4` configuration file (`/etc/proxychains4.conf`) must be updated to point to the local SOCKS listener. The entry socks4 127.0.0.1 8000 directs proxychains to send all application traffic through the SOCKS4 proxy on localhost port 8000, i.e., the active SSH dynamic tunnel.



```
root@kali ~  
➔ proxychains4 firefox  
[proxychains] config file found: /etc/proxychains4.conf  
[proxychains] preloading /usr/lib/x86_64-linux-gnu/libproxychains.  
[proxychains] DLL init: proxychains-ng 4.17  
[proxychains] DLL init: proxychains-ng 4.17  
[proxychains] DLL init: proxychains-ng 4.17  
[proxychains] DLL init: proxychains-ng 4.17  
[proxychains] DLL init: proxychains-ng 4.17  
[proxychains] Strict chain ... 127.0.0.1:8000 ... content-sigr  
[proxychains] Strict chain ... 127.0.0.1:8000 ... content-sigr  
[proxychains] DLL init: proxychains-ng 4.17  
[proxychains] DLL init: proxychains-ng 4.17
```

Launching Firefox Through Proxychains

Firefox is launched through proxychains using the following command, which intercepts all network calls and routes them through the configured SOCKS proxy.

```
proxychains4 firefox
```

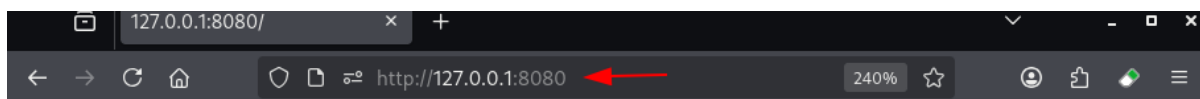
```
root@kali ~  
➔ proxychains4 firefox  
[proxychains] config file found: /etc/proxychains4.conf  
[proxychains] preloading /usr/lib/x86_64-linux-gnu/libproxychains.  
[proxychains] DLL init: proxychains-ng 4.17  
[proxychains] DLL init: proxychains-ng 4.17  
[proxychains] DLL init: proxychains-ng 4.17  
[proxychains] DLL init: proxychains-ng 4.17  
[proxychains] DLL init: proxychains-ng 4.17  
[proxychains] Strict chain ... 127.0.0.1:8000 ... content-sigr  
[proxychains] Strict chain ... 127.0.0.1:8000 ... content-sigr  
[proxychains] DLL init: proxychains-ng 4.17  
[proxychains] DLL init: proxychains-ng 4.17
```

The "Strict chain" messages indicate that all connections are being routed through `127.0.0.1:8000` — our SSH dynamic tunnel acting as a SOCKS proxy.

Accessing the Internal Service via Dynamic Proxy

With Firefox now sending traffic through the SOCKS proxy backed by the SSH tunnel, requests to 127.0.0.1:8080 are resolved from the remote SSH server's perspective rather than the local machine.





Welcome to Hacking Articles

In the proxied Firefox session, browsing to `http://127.0.0.1:8080` loads the “Welcome to Hacking Articles” page served by Apache2 on the remote loopback interface. From the browser’s standpoint, all requests are resolved on the SSH server, providing seamless access to the remote internal network.

Part 4: Remote Port Forwarding

Remote port forwarding (reverse tunnelling) is the counterpart to local port forwarding: it exposes a port on the remote SSH server and forwards it back to the local client or another host. This capability is highly valuable in penetration testing when a target sits behind NAT or a firewall and must safely expose an internal service to the attacker’s system.

Establishing the SSH Remote Port Forward

In this scenario, a Windows host establishes an SSH remote port forward to the Ubuntu SSH server, mapping port 3389 on the server’s loopback interface back to port 3389 on the local Windows machine for RDP access with help of the following command:

```
ssh -R 3389:127.0.0.1:3389 pentest@192.168.147.131
```



```
C:\Users\IEUser>ssh -R 3389:127.0.0.1:3389 pentest@192.168.147.131
pentest@192.168.147.131's password:
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-107-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

6 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

New release '24.04.4 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Fri Apr 10 15:01:23 2026 from 192.168.147.132
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-107-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

6 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

New release '24.04.4 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Fri Apr 10 15:01:23 2026 from 192.168.147.132
pentest@ignite:~$
```

This is a classic reverse tunnel pattern used to expose a Windows RDP service through an SSH pivot host.

Verifying the Remote Forward with Netstat on the SSH Server

On the remote SSH server (ignite), the following command is run to confirm that port 3389 is now listening on the loopback interface as a result of the reverse tunnel.

```
netstat -tnlp
```



```
pentest@ignite:~$ netstat -tnlp
(Not all processes could be identified, non-owned process info
will not be shown, you would have to be root to see it all.)
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 127.0.0.1:3389          0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.1:8080         0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:22             0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.53:53          0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.1:631          0.0.0.0:*               LISTEN      -
tcp6       0      0 :::1:631               :::*                    LISTEN      -
tcp6       0      0 :::22                  :::*                    LISTEN      -
tcp6       0      0 :::1:3389              :::*                    LISTEN      -
pentest@ignite:~$
```

The SSH server now listens for RDP connections on its loopback interface and tunnels them back to the originating Windows host, completing the reverse tunnel.

Part 5: RDP Access via Dynamic Port Forwarding and Proxychains

Dynamic SSH port forwarding establishes a local SOCKS proxy that routes all configured application traffic through the SSH tunnel. Combined with proxychains, this enables the RDP client to reach the Windows machine's RDP port via the Ubuntu SSH server acting as a proxy gateway.

Establishing the SSH Dynamic Tunnel from Kali

The first step is to set up a SOCKS proxy on the Kali machine by establishing an SSH dynamic port forward to the Ubuntu pivot host. The command for this is :

```
ssh -D 127.0.0.1:8000 pentest@192.168.1.9
```

```
root@kali ~
└─> ssh -D 127.0.0.1:8000 pentest@192.168.1.9
pentest@192.168.1.9's password:
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-107-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

6 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

New release '24.04.4 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Fri Apr 10 15:02:13 2026 from 192.168.147.132
pentest@ignite:~$
pentest@ignite:~$
```

While the session remains open, forward all SOCKS proxy traffic sent to Kali's port 8000 through the SSH tunnel. The Ubuntu host then evaluates it from its network perspective.



Launching rdesktop Through Proxychains

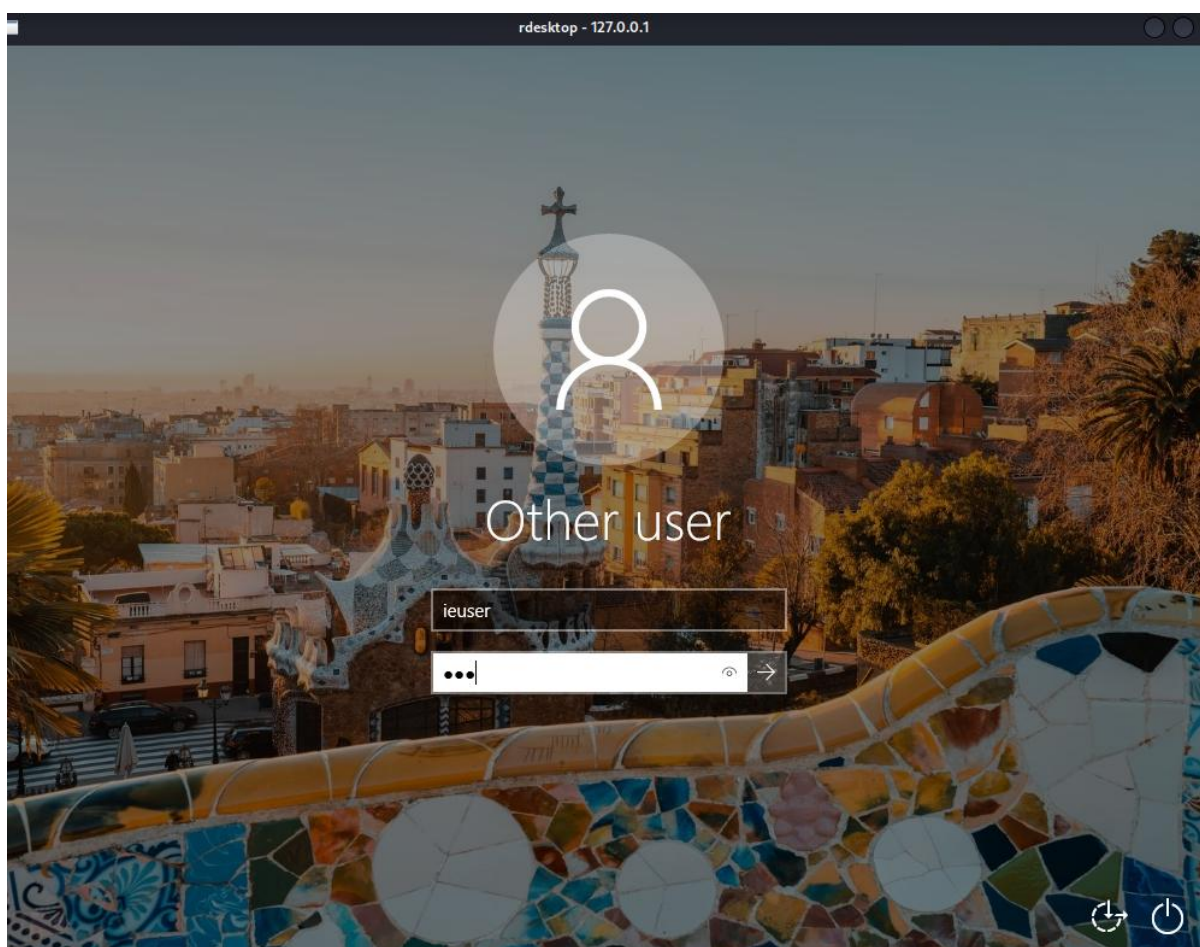
With the SOCKS proxy active, launch the rdesktop client through proxychains4. Proxychains4 intercepts its network calls and routes them via the SOCKS proxy at 127.0.0.1:8000, making the RDP connection appear to originate from the Ubuntu SSH server.

```
proxychains4 rdesktop 127.0.0.1
```

```
root@kali ~  
└─> proxychains4 rdesktop 127.0.0.1  
[proxychains] config file found: /etc/proxychains4.conf  
[proxychains] preloading /usr/lib/x86_64-linux-gnu/libproxychains.so  
[proxychains] DLL init: proxychains-ng 4.17  
Autoselecting keyboard map 'en-us' from locale  
[proxychains] Strict chain ... 127.0.0.1:8000 ... 127.0.0.1:3389  
Core(warning): Certificate received from server is NOT trusted by th  
Failed to initialize NLA, do you have correct Kerberos TGT initializ  
[proxychains] Strict chain ... 127.0.0.1:8000 ... 127.0.0.1:3389
```

Windows RDP Login Screen via Proxychains

With the proxied rdesktop connection established, the Windows RDP login screen appears for the Kali operator. The connection header confirms the session tunnels through the loopback address.

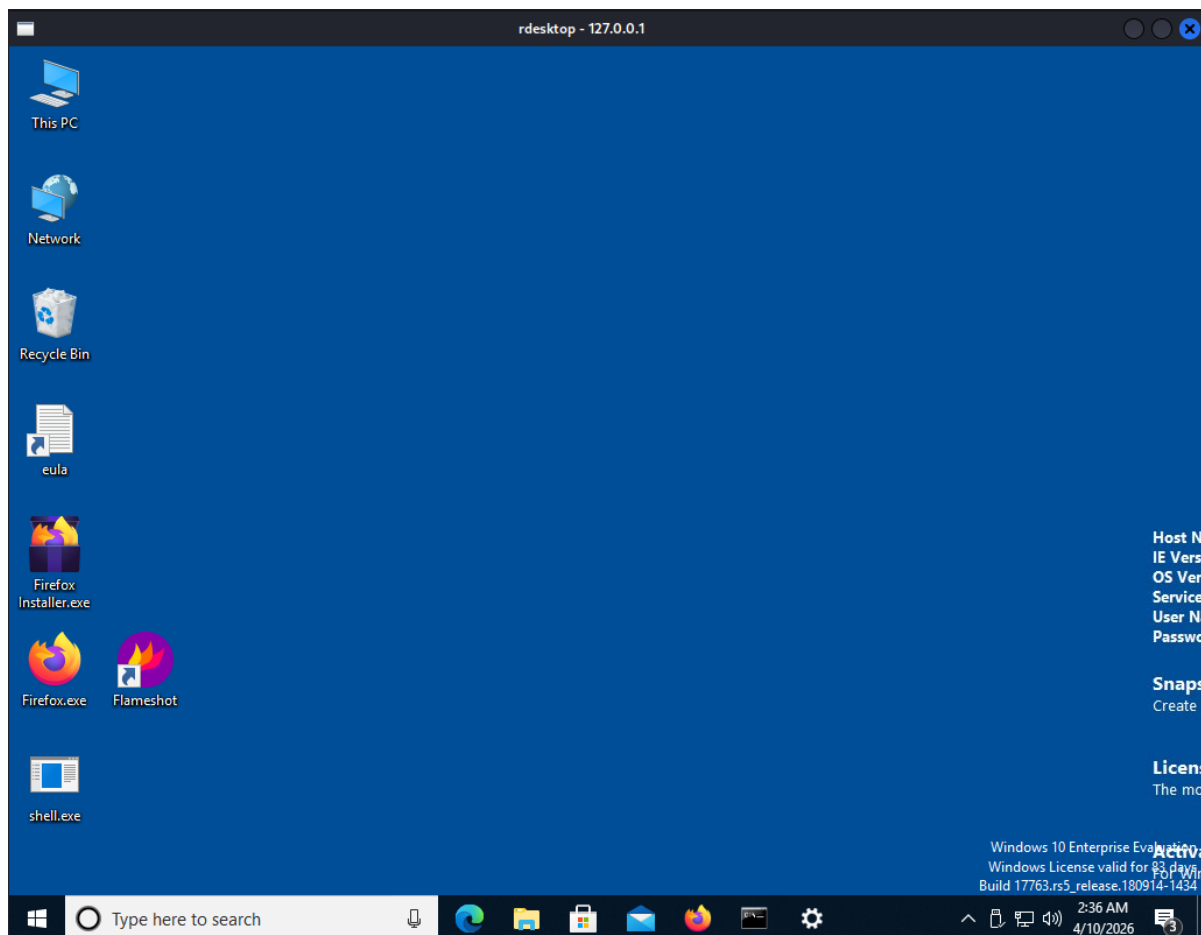




This demonstrates that the SOCKS proxy has successfully proxied the RDP traffic through the Ubuntu SSH server to the Windows target machine.

Accessing the Windows Desktop

After you enter the correct credentials, the full Windows 10 desktop appears, confirming complete RDP access through the SSH dynamic tunnel.



Part 6: RDP Access via SSH Remote Port Forwarding from the Pivot Host

In this scenario, the Ubuntu pivot machine (ignite) initiates an SSH remote port forward back to the Kali machine. This reverses the direction of the tunnel — instead of Kali connecting outward, the pivot machine pushes the RDP port back to Kali's loopback interface, making the Windows RDP service directly accessible on Kali's local port 3389.

Initiating Remote Port Forward from Ubuntu to Kali

The following command tells the Kali machine (192.168.1.17) to listen on its local port 3389 and forward incoming connections back through the tunnel to the Windows machine's RDP port.

```
ssh -R 3389:127.0.0.1:3389 kali@192.168.1.17
```



```
pentest@ignite:~$ ssh -R 3389:127.0.0.1:3389 kali@192.168.1.17
kali@192.168.1.17's password:
Linux kali 6.18.12+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.18.12-1kali1 (2026-02-25) x86_64

The programs included with the Kali GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Thu Apr  9 15:53:33 2026 from 192.168.1.9
(kali@kali)-[~]
```

The session authenticates successfully and establishes a shell on the Kali machine.

Verifying the Remote Forward on Kali with Netstat

Back on the Kali machine, netstat confirms that the Ubuntu pivot's inbound remote port forward now binds port 3389 on the loopback interface.

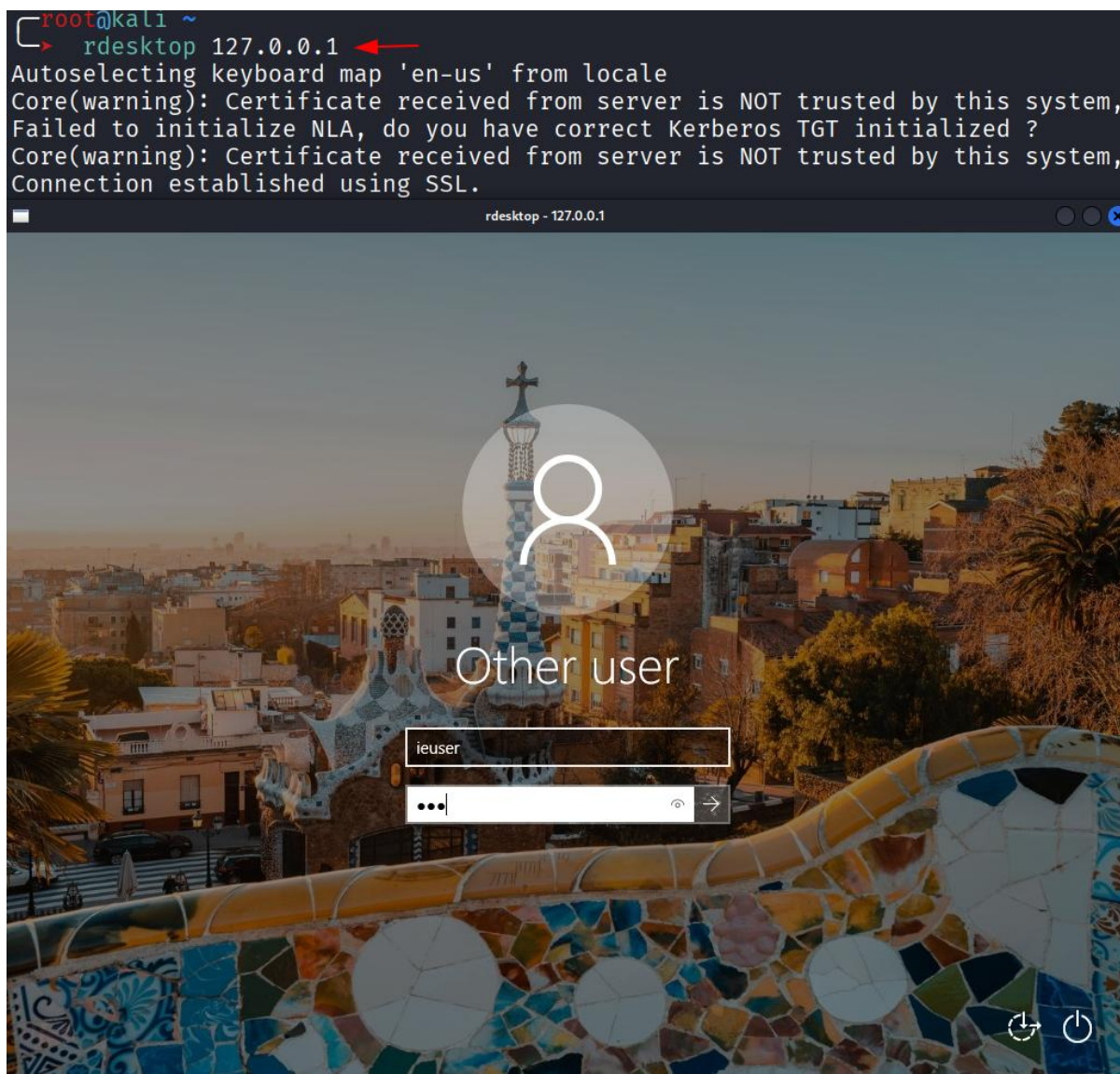
```
netstat -tnlp
```

```
root@kali ~
└─> netstat -tnlp
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 127.0.0.1:3389          0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.1:36941         0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN
tcp6       0      0 :::1:3389               :::*                     LISTEN
tcp6       0      0 :::22                   :::*                     LISTEN
```

Accessing RDP Directly on Kali's Loopback

With port 3389 now listening on Kali's loopback interface, use rdesktop directly — without proxychains. To initiate this use the following command:

```
rdesktop 127.0.0.1
```



The connection succeeds and the Windows RDP login screen appears within the rdesktop window.

Part 7: RDP Access via GatewayPorts — Exposing the Tunnel to the Network

The default behaviour of SSH remote port forwarding binds the forwarded port only to the loopback interface (127.0.0.1) of the SSH server — making it accessible only locally on that machine. The GatewayPorts directive in the SSH daemon configuration changes this behavior. It allows the forwarded port to bind on all interfaces (0.0.0.0), making the tunneled service accessible from anywhere on the network. This critical distinction matters in scenarios where teams need to share RDP access or multiple clients require it.

Checking the Default GatewayPorts Setting

Inspect the SSH daemon configuration file (/etc/ssh/sshd_config) on the Kali machine to review the default GatewayPorts setting before modification.



```
# and KbdInteractiveAuthentication to 'no'.
UsePAM yes

#AllowAgentForwarding yes
#AllowTcpForwarding yes
#GatewayPorts no
X11Forwarding yes
#X11DisplayOffset 10
#X11UseLocalhost yes
#PermitTTY yes
PrintMotd no
#PrintLastLog yes
#TCPKeepAlive yes
#PermitUserEnvironment no
#Compression delayed
#ClientAliveInterval 0
#ClientAliveCountMax 3
#UseDNS no
```

The directive `#GatewayPorts no` appears commented out with a hash (`#`), so the system applies the default value — `GatewayPorts` remains disabled. With this setting, any remote port forward received by this SSH server will only bind to the loopback interface, not the network-accessible interfaces.

Enabling GatewayPorts

To make the forwarded port accessible from the network (not just loopback), explicitly enable the `GatewayPorts` directive by editing `sshd_config` and setting it to "yes".

```
# and KbdInteractiveAuthentication to 'no'.
UsePAM yes

#AllowAgentForwarding yes
#AllowTcpForwarding yes
GatewayPorts yes
X11Forwarding yes
#X11DisplayOffset 10
#X11UseLocalhost yes
#PermitTTY yes
PrintMotd no
#PrintLastLog yes
```

With this setting active, the SSH daemon will bind incoming remote port forwards to all network interfaces (0.0.0.0) rather than only the loopback address, making the forwarded RDP port reachable from any machine on the network.



Restarting the SSH Service

For the GatewayPorts configuration change to take effect, you must restart the SSH daemon on the Ubuntu pivot machine. The following command will gracefully restarts the SSH daemon, loading the updated sshd_config with GatewayPorts enabled.

```
pentest@ignite:~$  
pentest@ignite:~$ sudo service ssh restart  
pentest@ignite:~$
```

Initiating Remote Port Forward from Windows to Ubuntu

With GatewayPorts now enabled on the Ubuntu pivot, the Windows machine initiates the SSH remote port forward. This pushes the Windows RDP port (3389) to the Ubuntu SSH server's loopback, and because GatewayPorts is enabled, it binds on all interfaces. And to do so, use the following command:

```
ssh -R 3389:127.0.0.1:3389 pentest@192.168.1.9
```

```
C:\Users\IEUser>ssh -R 3389:127.0.0.1:3389 pentest@192.168.1.9  
pentest@192.168.1.9's password:  
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-107-generic x86_64)  
  
 * Documentation:  https://help.ubuntu.com  
 * Management:    https://landscape.canonical.com  
 * Support:       https://ubuntu.com/pro  
  
Expanded Security Maintenance for Applications is not enabled.  
  
0 updates can be applied immediately.  
  
6 additional security updates can be applied with ESM Apps.  
Learn more about enabling ESM Apps service at https://ubuntu.com/esm  
  
New release '24.04.4 LTS' available.  
Run 'do-release-upgrade' to upgrade to it.  
  
Last login: Fri Apr 10 15:39:05 2026 from 192.168.1.3  
pentest@ignite:~$ netstat -tnlp  
(Not all processes could be identified, non-owned process info  
will not be shown, you would have to be root to see it all.)  
Active Internet connections (only servers)  
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name  
tcp        0      0 127.0.0.1:8080          0.0.0.0:*                LISTEN      -  
tcp        0      0 0.0.0.0:22              0.0.0.0:*                LISTEN      -  
tcp        0      0 0.0.0.0:3389            0.0.0.0:*                LISTEN      -  
tcp        0      0 127.0.0.1:631           0.0.0.0:*                LISTEN      -  
tcp        0      0 127.0.0.53:53           0.0.0.0:*                LISTEN      -  
tcp6       0      0 :::22                   :::*                    LISTEN      -  
tcp6       0      0 :::3389                  :::*                    LISTEN      -  
tcp6       0      0 :::1:631                 :::*                    LISTEN      -  
pentest@ignite:~$
```

After the Windows host authenticates to the Ubuntu SSH server, netstat -tnlp on the ignite machine shows 0.0.0.0:3389 in the LISTEN state, confirming that GatewayPorts is correctly



exposing the forwarded RDP port on all interfaces and making it reachable across the network.

Direct RDP Connection from Kali Using the Network IP

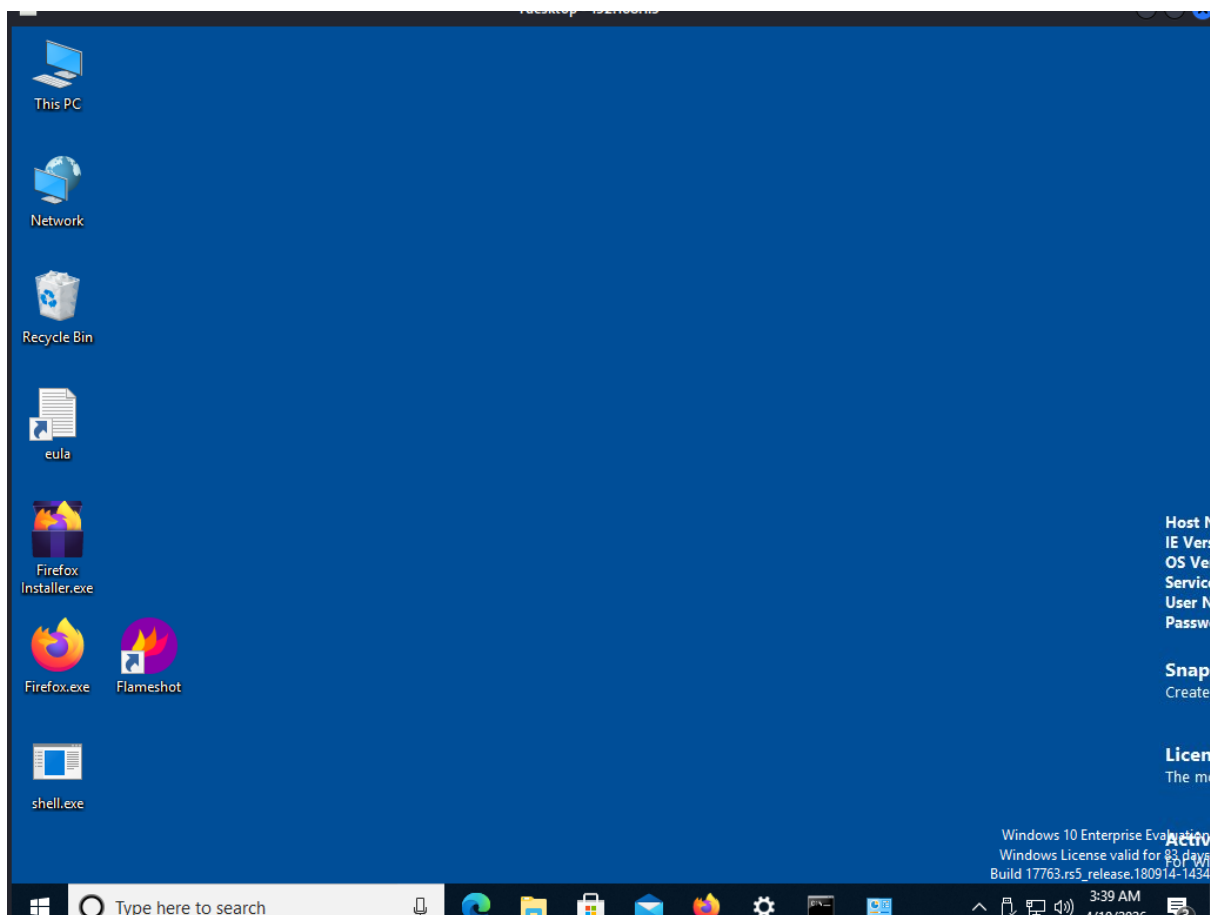
With the RDP port now bound to 0.0.0.0:3389 on the Ubuntu pivot, the Kali machine can connect to RDP using the Ubuntu server's actual network IP address — no tunneling, proxychains, or loopback tricks required from Kali's side. Just use the command:

```
rdesktop 192.168.1.9
```

```
root@kali ~  
└─> rdesktop 192.168.1.9  
Autoselecting keyboard map 'en-us' from locale  
  
ATTENTION! The server uses an invalid security certificate which can not be trusted for the following identified reason(s):  
  
1. Certificate issuer is not trusted by this system.  
  
    Issuer: CN=MSEDGEWIN10  
  
Review the following certificate info before you trust it to be added as an exception:  
If you do not trust the certificate the connection attempt will be aborted:  
  
    Subject: CN=MSEDGEWIN10  
    Issuer: CN=MSEDGEWIN10  
    Valid From: Fri Apr  3 00:13:33 2026  
    To: Sat Oct  3 00:13:33 2026  
  
Certificate fingerprints:  
  
    sha1: 9e79ad87f6021dcdf8b69aa7c32c9df14afe50c9  
    sha256: 635223d6171f5c6c0dde5cbfcaa9d02abd37a48c0f752539c5e3415e84ad80a4
```

Full Windows Desktop via GatewayPorts RDP Tunnel

The final proof of concept: the GatewayPorts-enabled remote port forward renders a full Windows 10 desktop on the Kali machine, connected via the Ubuntu pivot's network IP.



In this case, the GatewayPorts-enabled SSH remote port forward delivers the RDP session. The rdesktop title bar shows a connection to 192.168.1.9, the Ubuntu pivot's IP, while rendering the Windows 10 desktop, confirming full graphical access. This pattern is especially effective in collaborative red team operations where multiple operators require concurrent RDP access through a single pivot host.

Conclusion

This article highlights SSH port forwarding as both a legitimate network engineering capability and a highly effective technique for penetration testers. Across six practical scenarios, it demonstrates how attackers can remotely access services bound solely to a loopback interface through various SSH tunnelling methods, without deploying extra agents or third-party tools on the target.

The four SSH forwarding modes—Local (-L), Dynamic (-D), Remote (-R), and GatewayPorts-enabled Remote—serve different access needs. Local forwarding suits direct SSH access to a pivot and a single internal service. Dynamic forwarding with proxychains extends this to arbitrary application traffic via a SOCKS proxy. Remote forwarding reverses the flow, allowing a host behind NAT or a firewall to expose its internal services to an external listener. With GatewayPorts enabled, those forwarded ports become reachable across the entire network, supporting multi-operator red team operations.



Together, these techniques demonstrate that SSH access to any host can act as a gateway into the wider internal environment. Achieving full graphical RDP access to a hardened Windows 10 Enterprise system required only standard SSH and rdesktop. As a result, detecting and controlling SSH tunnels must be a core function of security operations, with SSH treated as a privileged channel and monitored accordingly.

Mitigation Strategies

Detecting and controlling SSH tunneling is as important as understanding how it works. The following defense-in-depth measures help reduce the risk of abuse in enterprise environments.

First, restrict SSH exposure and harden authentication. Limit SSH access to approved users and hosts using firewall rules, allowlists, and host-based controls. Disable password authentication in favor of key-based logins, and enforce multi-factor authentication to significantly raise the bar for unauthorized tunnel creation.

Second, explicitly lock down forwarding capabilities. In the SSH server configuration, disable unneeded options such as TCP forwarding, reverse forwarding, and tunneling features, and ensure GatewayPorts remains off unless there is a clear, documented business need. Regularly review these settings as part of configuration audits.

Third, deploy strong network-level monitoring and analytics. Although SSH encrypts payloads, security teams can still detect suspicious behavior by monitoring for unusually long-lived SSH sessions, abnormal data volumes, and connections from unexpected sources. These patterns should feed into SIEM and network analysis platforms, with well-defined alerts and SOC playbooks.

Finally, align controls with a Zero Trust approach. Treat SSH as a highly privileged channel, enforce just-in-time access, use hardened administrative workstations, and record sensitive sessions for auditing and rapid incident response.

FOLLOW US ON

social media



TWITTER



DISCORD



GITHUB



LINKEDIN

CONTACT US
FOR MORE DETAILS

+91 95993-87841

www.ignitetechnologies.in

JOIN OUR TRAINING PROGRAMS

